Workload identity (federated agents) — setup guide

> Verified live end to end **2026-11-13** on gateway `mcpgw.ai-demo.ping-devops.com`

> (`privilege-aigw v1.261024+`), PingFederate issuer `https://cmuir-pf.ping-devops.com`,

> Privilege tenant `0428ba4f-…`. Every step below was re-run and returned 200 today.

What this proves

An AI agent authenticates to the Privilege gateway with a **JWT issued by the customer's own identity platform **(here PingFederate) —** no Privilege-issued secret exists for the agent at all**. The gateway verifies the token cryptographically against the issuer's JWKS, matches issuer/audience/required claims, and forwards the token unchanged to the backend (Pass Agent Token).

This is the federated-identity story for AI platform agents: identity stays in the customer's IdP; Privilege governs what the agent may do, without minting or storing a key for it.

Covered	How
Agent authenticates with an external IdP token	Non-Human Identity trusts the issuer + JWKS
No Privilege secret in the agent	The IdP token is the only credential
Only one specific agent accepted	Required claims `act.sub` + `client_id`
Token bound to one agent app	`aud` = the agent app URL
Tools per grant	The agent key's grant (or scope tiers with `RequiredClaims`)
Backend verifies the token itself	Pass Agent Token forwards the token unchanged

Architecture

```
agent ■■1■■■ PingFederate (client_credentials) ■■■ JWT (iss/aud/act.sub/scope)
  ■
  ■■2■■■ Privilege gateway  POST /pftoken/mcp  Authorization: Bearer <JWT>
              ■ 3. verify signature vs issuer JWKS; match iss/aud/RequiredClaims
              ■ 4. apply agent key's grant (tools) + Cedar policies (scopes)
              ▼
          MCP backend  ■■■ 5. token forwarded unchanged (Pass Agent Token)
```

The two key kinds side by side:

Kind	Caller sends	Gateway check	Where issued
Virtual key (Agent Registry)	secret string as Bearer	lookup + scope + owner active	Privilege console
Workload identity	**JWT from caller's own platform**	**signature vs registered JWKS, issuer/subject match**	**customer IdP (PF, K8s, GitHub Actions, SPIFFE, any OIDC)**

Prerequisites

- Gateway `privilege-aigw:v1.261024` or later (earlier versions cannot discover tools on a Pass Agent Token agent app).
- Admin access to PingFederate (13.1) and the Privilege console.
- The agent app's backend reachable from every gateway pod.

Setup, part A — PingFederate: make the agent's token

1. **OAuth client** for the agent, grant `client_credentials` (scopes unrestricted or limited to what you need). [`pf-route-c-test` — "Privilege PF token demo agent"]
2. **Scopes** on the authorization server: whatever your tiers/Cedar will check. [`openid opensearch:read opensearch:admin whoami:read`]
3. **JWT access token manager** (clone an existing JWT one; unique Key ID):
- Audience claim value = the agent app URL. [`https://mcpgw.ai-demo.ping-devops.com/pftoken/mcp`]
- Resource URIs = the same URL.
- Scope Claim Name = `scope` (default). Do **not** add `scope` to the attribute contract — the manager writes granted scopes itself.

[`routeCTestAtm` — "Privilege PF token demo access tokens", signing cert `aidemopf-idjag-signing`, kid `aidemopf-idjag-signing-routec`, RS256]
4. **Access token mapping** for the `CLIENT_CREDENTIALS` context on that manager:
- `sub` ← Context: ClientId
- `client_id` ← Context: ClientId
- `act` ← Expression `#{"sub": "<agent id>"}` — must be an **object**, not text

[`id: client_credentials|routeCTestAtm`]
5. Set the client's **default access token manager** to the one from step 3.
6. **Every token request must send** `resource=<agent app URL>` — otherwise a scoped request can be served by a different token manager (wrong `aud`, no `act`) and the gateway answers 401 `token_not_found`.
7. **Check**: mint a token and decode the payload. You need `iss`, `sub`, `aud` = agent app URL, `client_id`, `act.sub`, `scope`.

Setup, part B — Privilege: register the federated agent

Console: **Non-Human Identities** → **Register Identity** (Ping's 2026-11-02 deck renames this to **Federated Identity**; the API object is still `NonHumanIdentity` at `/v1/nonhumanidentities`).

1. Token issuer type: **OIDC Provider**.
2. **Issuer (`iss`)**: exactly the token's `iss`. [`https://cmuir-pf.ping-devops.com`]

3. **Key source**: Fetch from URL; JWKS URL = the IdP's JWKS.

[`https://cmuir-pf.ping-devops.com/pf/JWKS`]

4. **Subject**: `*` (the agent is pinned by required claims instead).

5. **End user in a presented token**: Opaque (or Resolved if the end user exists

in Privilege — then per-user Cedar rules apply).

6. **Audience**: the agent app URL — **generated display vs the token must match exactly**.

7. **Required claims**: `act.sub` = `<agent id>` and `client_id` = `<client id>`.

[`both pf-route-c-test`]

8. **Permissions/scopes**: `mcp` (add `llm`/`a2a` only if needed).

9. Save. If the console refuses with "Spec.UserClaim resolves to sub…", set

Subject to `*` as in step 4.

Setup, part C — Privilege: the agent app

Console: **Agentic Apps** → **Add Application** → **MCP Server**.

1. **MCP Server URL**: reachable from every gateway pod, ending in exactly `/mcp`

(the URL path becomes the app's entry path; a saved `…/mc` gives 404

`outside entry path`). [`http://opensearch-mcp-noget...svc.cluster.local/mcp`]

2. **Auth Mode: Pass Agent Token.**

3. Mesh cluster = your gateway's. Save — Overview shows "Awaiting first agent

call - tools will be discovered from a forwarded assertion".

Setup, part D — Privilege: the agent key and its tools

Console: **Agent Registry** → **Add Agent**:

1. **Custodian: the identity from part B.** Scope: `mcp`. [`key pftoken-agent`]

2. Gateway & Apps: the agent app from part C.

3. Configure Tool Access: tier Everyone, select the tools (none selected = all).

4. Save. Do **not** create the grant as a separate API policy — the gateway

deletes those within seconds; edit the key's own grant instead.

Setup, part E — first call and verification

1. One agent call (`tools/list`) triggers tool discovery even if refused.

2. Call again: **200** with the tool list; a garbage bearer still gets **401**.

3. Gateway log: `workload token accepted for key <key> issuer=<iss> subject=<sub>`,

no `policy denied`.

Live verification 2026-11-13 (all commands re-run)

Check	Result
Mint PF token (client_credentials + resource=)	token with `iss/aud/act.sub/scope` ✓
`POST /pftoken/mcp tools/list` with PF JWT	**HTTP 200** — ListIndexTool, SearchIndexTool, CountTool
`POST /pftoken-rs/mcp tools/call whoami`	**HTTP 200** — backend reported the **unchanged PF token** (issuer, `act.sub`, scopes)
Control: garbage bearer	**401 ✓**
Privilege NHI `PFtoken`	enabled, issuer/JWKS/audience/required-claims intact

Scripts

- `privilege/scripts/workload-identity-mint.sh` — mints the PF token, decodes the claims, prints them. Secret read from env `PF_ROUTE_C_SECRET` (never hardcoded, never echoed).
- `privilege/scripts/workload-identity-call.sh` — full demo: mint → `tools/list` on the agent app → `whoami` on the passthrough backend → garbage-token control. Prints only outcomes and claims, never the token.

Curl quick reference

```
# 1. Mint the federated token (PingFederate client_credentials)
curl -s https://<pf-host>/as/token.oauth2 \
  -u "<client id>:$PF_CLIENT_SECRET" \
  -d grant_type=client_credentials \
  -d "scope=openid opensearch:read whoami:read" \
  -d "resource=https://<gateway-host>/<agent app>/mcp"

# 2. Call the gateway with the workload token (no Privilege secret anywhere)
curl -s https://<gateway-host>/<agent app>/mcp \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json, text/event-stream' \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'

# 3. Prove passthrough: the backend sees the token unchanged
curl -s https://<gateway-host>/<rs agent app>/mcp \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -H 'Accept: application/json, text/event-stream' \
  -d '{"jsonrpc":"2.0","id":2,"method":"tools/call","params":{"name":"whoami","arguments":{}}}'
```

Failure reference

Symptom	Cause	Where to look
401 `token_not_found`	request lacked `resource=`, or wrong token manager served it	PF: default ATM on the client; decode token's `aud`
401 `workload token rejected`	signature/issuer/audience/required-claim/lifetime mismatch	gateway log names the reason
403 after token accepted	no grant rule matched the app (or no scope tier matched)	gateway log `policy denied`
404 `outside entry path`	agent app URL doesn't end in `/mcp`	console app config

Symptom	Cause	Where to look
200 with no tools	tools not discovered yet or backend unreachable	gateway log `tool discovery … failed`
403 `requires token-pass, but this session presented no workload assertion`	app on Pass Agent Token but caller is a person or virtual-key-only agent	call from an agent holding an IdP token
Claims check fails but token looks right	`act` mapped as text not object, or `scope` added to the attribute contract	PF token manager mapping

Secret hygiene

- The PF client secret lives only in your local store (`PF_ROUTE_C_SECRET` env or the BFF's config store). It is never printed, never committed, never in the guide. Rotate it in the PF admin console/API (`PUT /pf-admin-api/v1/oauth/clients/<id>` with `clientAuth.secret` on the PingFederate admin API); the gateway needs no change — it holds no secret for the agent at all, which is the point.

Relationship to the other lanes

Lane	Credential	Governed by
Models as REST (`/llm/*`)	virtual key (Agent Registry)	key: provider binding, model allow-list, rates, budget, AI Guard
Platform agents (MCP door)	agent key or **workload JWT**	key grant + Cedar
Agent-to-agent (A2A, `token-pass`)	caller's **workload token forwarded**	remote agent checks identity end to end

Workload identity is the federated member of the key family: same grant/policy machinery, but the credential comes from the customer's IdP, so agent onboarding needs zero secret exchange.